

Evidence-Bound Recovery for Ambiguous Submissions in Local Generative-AI Pipelines

Tool Description

Anonymous submission draft

Abstract—A lost acceptance response leaves a generation client uncertain about backend execution. Our tool preserves reported state, interprets evidence through a read-only exporter, and blocks further submissions within an unresolved run. Four historical Windows/ComfyUI cases retained one execution binding per control and two per response-loss case. Their retries created fresh runs outside the guard. A subsequent same-run test stopped at model-identity validation. After the caller rebuilt pinned process-local inventory, a revised test reached the submission-outcome-unknown guard. It added no submission and left the original run unresolved. Synthetic CPU fixtures show an incomplete client operation despite worker completion. Retained raw payloads support the new lifecycle bindings; the historical event gap remains. The fixed cases demonstrate evidence export and run-scoped submission control. They establish no automatic reconciliation, deployment rates, physical GPU savings or output-quality gains.

I. INTRODUCTION

An image-generation client can lose a response after its backend accepts the workflow. Without the returned job identifier, the client cannot determine whether resubmission will repeat completed work. Request counts and timeouts alone leave that question unanswered. Recovery therefore requires an explicit account of the evidence available to the client and the backend.

Remote procedure call semantics already address uncertainty under communication failure [1]. Idempotent APIs make retries safe through request identifiers and an explicit service contract [2]. Local execution leaves the same distinction between transmitting a request and observing its outcome. Our tool records client state and backend observations separately in a generation pipeline; it adds no new exactly-once protocol.

The prototype combines persistent run records, a read-only evidence exporter, and a guard that blocks submissions through an explicitly unresolved run. Separate backend observations constrain the exporter’s versioned interpretation of client outcomes. CPU fixtures and Windows campaigns demonstrate these interfaces under controlled response loss. Blocking a submission leaves the original run’s recovery obligation unresolved. We contribute a local implementation that connects the run records, export decisions and submission checks, with explicit failure boundaries. Section VI compares these responsibilities with existing control and tracing contracts.

II. FAILURE MODEL AND EVIDENCE SEMANTICS

A **product call** is an invocation by the research client. A **run** is the persistent product object created by

`local_gpu_start_run`. A **submission** in the Windows results is a proxy-observed `POST /prompt`; a call that fails before that boundary need not produce a submission. An **execution instance** is an observation bound to backend execution events. CPU fixtures instead observe the synthetic worker entry. These units are not interchangeable.

F00 is a normal control. F02 suppresses the first accepted submission response, creating a window in which the client lacks an acknowledged job identifier. F03, covered by the CPU fixtures rather than the Windows campaign reported here, concerns loss of completion information after a job identifier is known. An explicit rejection and a missing response have different evidential meanings. Neither should be silently converted into a backend execution outcome.

The evidence model distinguishes reported state, interpreted state and oracle state. In addition, the Windows research client emits its own call-level labels. Its `resolved` label is derived from a generated product result after checking for artifact hashes. Its `unresolved` label can arise from several client errors. A research-client label alone does not establish a durable product manifest’s recovery state. In F02, the proxy’s accepted-response loss and the oracle binding provide the additional context.

III. TOOL DESIGN

A. Evidence-preserving export

The exporter accepts a record or run manifest and produces a separate normalized record. It copies reported state, records the mapping version and reasons, and retains supplied oracle state separately. The schema includes source identity, request and input digests, backend identity, job identifier, artifact hash and validator version. Missing fields carry explicit reasons instead of fabricated values. The inspected mapping is `research-normalization-v4`.

Interpretation checks more than an execution-success label. Job and artifact bindings, approval and validation evidence, and recovery state constrain the `execution_verified` decision. An unresolved recovery state prevents this flag from becoming true even when other evidence reports successful execution. The current mapping also keeps recovery unknown when the reported run state is missing and marks a disagreement between manifest and selected job or artifact identity as an evidence mismatch. These rules prevent omission or a top-level field from silently strengthening the composite decision.

A record can therefore report backend completion while leaving the product’s recovery obligation unresolved. The exporter does not invoke the engine, retry a request or update the source run. Its output writer uses exclusive file creation. These are implementation properties, not a claim of complete filesystem security. The interpretation assumes that supplied oracle and artifact-validator records have trustworthy provenance. The exporter checks their stated fields and bindings; it does not authenticate their origin or reconstruct missing raw events. In the current mapping, absent source revision, backend-instance or validator-version metadata need not by themselves prevent `execution_verified`. That flag denotes satisfaction of this mapping’s evidence policy, not independent certification of the input record.

B. Using the evidence record

An investigator first inspects the run manifest and the recorded submission outcome. If submission is explicitly unknown, inspection must preserve that uncertainty: the missing job identifier must not be synthesized from a later successful call. Backend observation is then associated with the relevant request and execution instance, while artifact checks retain their own result. The export command writes a new record and refuses to overwrite an existing export:

```
python -m
scripts.research.export_records
-input run-record.json -output
evidence.json
```

The output exposes reported state, interpretation, mapping reasons and supplied oracle evidence without initiating recovery. Missing oracle evidence remains missing. The Windows observer and exporter use separate schemas; the offline conversion described below bridges retained inputs rather than providing automatic live conversion. The companion offline demonstration supplies two synthetic records that differ in the original run’s status. It also normalizes twelve previously retained CPU product manifests alongside their fixture oracle records, without adding artifact-validation assertions. All twelve remain unverified by the composite policy. This demonstrates the mapping’s behavior on retained product records, not an independent replay or certification of their original events.

The exported record can contain successful backend evidence, required recovery and false execution verification. A later successful call leaves the original run’s unresolved state intact.

C. Recovery guard and its scope

`AssetRunEngine` and `RunStore` preserve the explicit transport marker `submission_outcome=unknown` as an unresolved attempt. The reviewed submission entry checks that unresolved attempt before permitting further generation. The guard applies within the same run, including attempts with a different idempotency key. The corresponding inspection action is `get_run`.

The guard neither discovers a missing job identifier nor reconciles an unknown job. A new run lies outside its scope.

Known-job recovery in two-stage operations uses a separate path. Withholding another submission leaves the outcome uncertain and supplies no completion guarantee.

A pipeline integrator can use this distinction when a generation UI receives a disconnect after sending a request: the exporter preserves the unknown outcome, the same-run entry blocks another submission, and the operator must inspect or reconcile the run through an available backend-specific mechanism. The guard can also conservatively withhold work when a request failed before backend acceptance, which is a potential false-positive direction. In the accepted-response-loss CPU fixture, the observed cost is an incomplete client operation; eventual completion is not established.

D. Controlled observation on Windows

Figure 1 shows the product and research boundaries. The research harness adds a transport shim and loopback fault proxy. The shim routes prompt transport through the proxy while retaining the actual backend endpoint in the product’s identity client. F02 drops the first response only after backend acceptance. This arrangement is a research instrument, not a native ComfyUI recovery feature.

The observer connects before the product invocation and uses the operation key as its ComfyUI client identifier, matching the product’s event-routing identity. `ComfyUIEventOracle` requires an execution-start event, a terminal event and completed history bound to a prompt identifier. It derives an execution-instance identifier using the backend boot identity, prompt identifier and observed start sequence. Its independence is from the product return value: it still trusts the backend’s event and history interfaces, and it is not a physical GPU monitor or a crash-durable audit log.

The controller permits the second F02 product call only after the first execution has a bindable completion. In this historical protocol, the client creates a new run on each call and uses `seed = 4100 + call_index`. This is observation-gated fresh-run generation, not automatic reconciliation of the original run. Because the new run is outside the same-run guard, the Windows campaign does not measure the guard’s blocking effect. CPU F02 fixtures exercise blocking; in the single-stage fixture, W3 has one execution rather than B2’s two, while leaving the operation incomplete. Matching frozen request digests does not establish full prompt equality: the inspected digest function excludes the seed and does not use its plan argument.

IV. EVIDENCE AND DEMONSTRATION

A. CPU components and deterministic fixtures

We denote the frozen baseline product revision by B2 and the revision with the same-run ambiguity guard by W3. The CPU harness separates submission events from synthetic worker-entry events. Its fixtures cover normal operation and response loss before and after job-ID acquisition, including single-stage and two-stage paths. Oracle self-checks exercise cases in which submission and execution counts differ and in which a shared prompt identifier does not imply a single

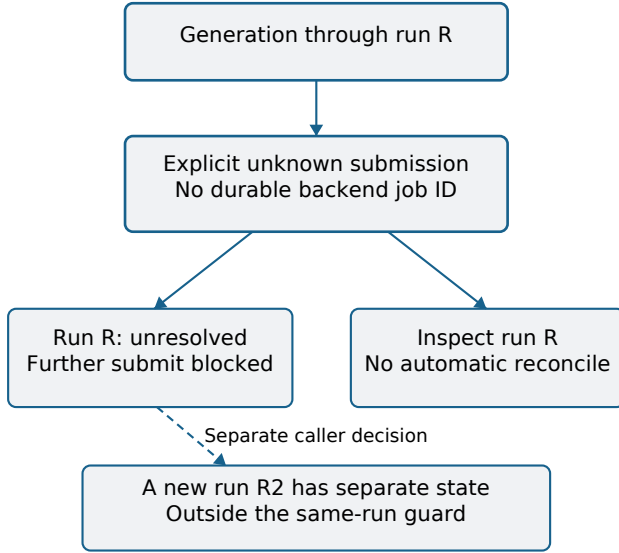


Fig. 1. Scope of the product guard, derived from the reviewed store and engine entry points. A new run has separate state. Dashed arrows denote a separate caller decision, not automatic reconciliation.

execution. These are fixed semantic checks, not randomly sampled deployments.

The delivered offline checks were tested with Python 3.13.15. Their source requires Python 3.10 features; compatibility with 3.10 itself was not tested. They check retained-record consistency and mapping, not backend execution. Historical test-count and runtime claims without packaged receipts are omitted. Exact historical CPU test-worktree attestation remains unavailable. The Windows v1/v2 campaigns recorded nine harness source hashes each. Separate source snapshots reconstructed from the respective Git revisions with Windows CRLF bytes match all eighteen recorded hashes. A packaged verifier checks this correspondence; byte matching is not runtime execution attestation. The current research caller is not substituted for either execution snapshot.

Table I reports the retained paired CPU matrix. Its recovery labels are historical fixture outcomes, not exporter enumerations. In normalization v4, a `partial` reported run maps to unknown recovery, rather than no recovery requirement; this corrects the B2 F02 two-stage interpretation without changing any execution or submission count. Reanalysis checked all twelve synthetic traces: each execution-start identifier was unique within its case and matched a finish identifier, and request-received counts matched the stored submission totals. The control cases are separate from injected cases. The invalid earlier matrix version, whose single-stage control failed because of a harness result-field mismatch, is excluded.

In the F02 single-stage fixture, W3 blocks a second product submission. Its synthetic worker records one successful start and finish with an artifact, yet the product operation remains incomplete in the observation window. B2 submits twice and completes in this fixture. The guard’s direct action is withhold-

ing a submission; the corresponding difference in synthetic execution counts is not a measured real-backend reduction. The unchanged F03 two-stage row exercises existing known-job recovery. These CPU paths must not be pooled with the Windows fresh-run protocol in Fig. 2.

B. Four controlled Windows cases

The endpoint-preserving campaign report records B2 and W3 under F00 and F02. All four fixed cases were oracle-evaluable; the two F00 cases were controls and the two F02 cases had confirmed response-loss injection. Table II reports the observations without converting these counts into a success probability.

The campaign totals were six calls, six proxy submissions and six retained bound execution instances. The F02 second call used a fresh run and changed seed. The four report records matched the corresponding retained JSONL cases. Their six retained execution bindings had distinct identifiers, completed-history flags and terminal-event labels. In both B2_F02 and W3_F02, the export reports `execution_start=3`, two proxy submissions and two retained bindings.

Each F02 case has two observer decisions. The oracle’s `raw_events` field is cumulative, while the export counts the entire event list of each decision. For example, if the first list contains start A and the second contains A and B, the sum is $1 + 2 = 3$ entries for two distinct starts. Each F00 case has only one decision, so this recounting opportunity is absent there. The uniqueness check concerns the two retained binding identifiers per F02 case, not the three aggregate entries. This source-level mechanism makes the reported count compatible with two bindings; it does not reconstruct the historical lists. Because the original snapshot boundaries and payloads are unavailable, an additional unbound start cannot be independently excluded. We report two retained bindings per case, not a complete execution ground truth.

The first binding’s completion precedes the second receipt, as required by the controller’s observation gate; these timestamps belong to different calls. No rate is estimated. Here, “oracle-evaluable” refers to the retained binding record, not an independent raw-event audit.

The report identifies an RTX 5070 Ti Laptop GPU and ComfyUI v0.30.0 at a pinned source revision, with model and workflow hashes checked against the campaign configuration. These are recorded environment facts for this campaign. They do not establish behavior across other models, backend versions or machines.

The research export retained execution identifiers despite the first call’s ambiguous outcome. The cases provide no execution-count advantage for W3. Changing both run and seed also prevents interpreting them as repeated executions of an identical generation request.

C. Same-run scope tests and a recovered guard observation

A subsequent campaign retained the frozen W3 product but used a separate `same-run-guard-v1` research caller. F00 invoked one generation call. F02 saved the first run and

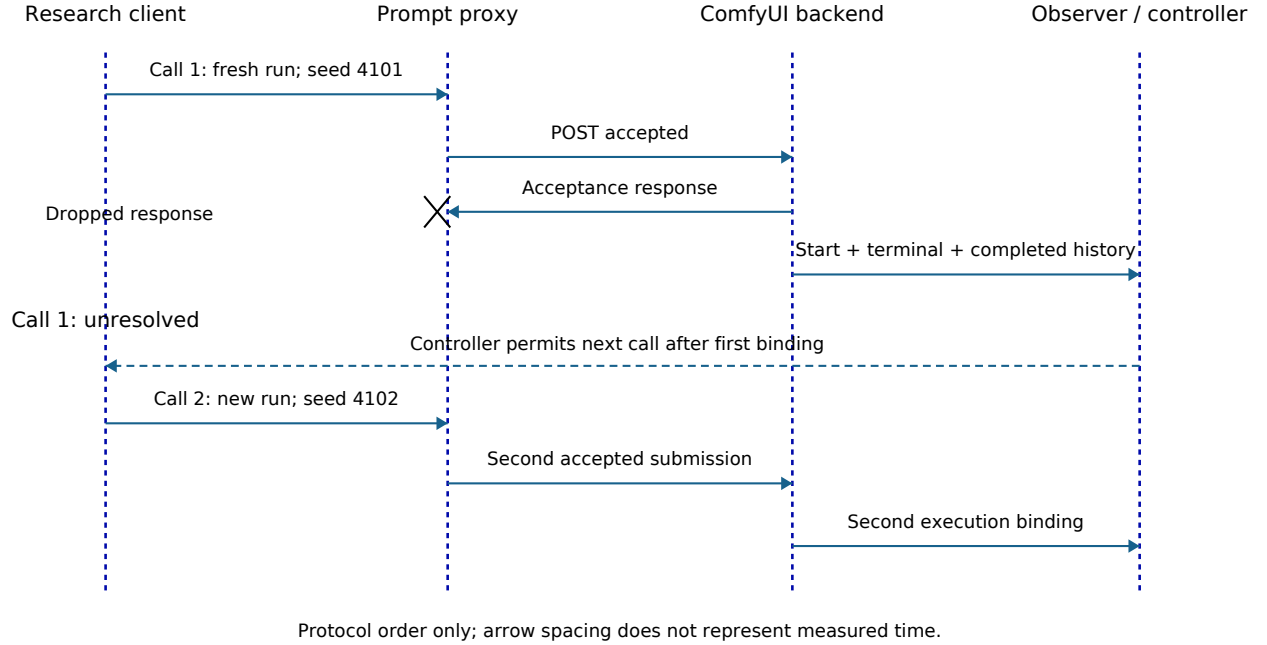


Fig. 2. Controlled F02 protocol. The proxy hides the accepted response; backend observation gates a second call that creates a fresh run with a changed seed. Retained receipts and bindings support the two submissions and the ordering of the second receipt after the first retained binding. The campaign does not exercise the same-run guard. Spacing is schematic, not a measured latency or GPU-runtime plot.

TABLE I
DETERMINISTIC CPU FAULT COVERAGE FROM THE RETAINED PAIRED-V2 MATRIX. S/E MEANS SUBMISSIONS/EXECUTIONS. “COMPLETE” IS THE FIXTURE’S VALID-COMPLETION FLAG. THESE ARE TWELVE SYNTHETIC TRACES, NOT DEPLOYMENT SAMPLES.

Fault	Path	B2 S/E	W3 S/E	Complete (B2/W3)	B2 recovery	W3 recovery
F00	Single stage	1/1	1/1	yes/yes	Not needed	Not needed
F00	Two stage	1/1	1/1	yes/yes	Not needed	Not needed
F02	Single stage	2/2	1/1	yes/no	New submission	Unknown; blocked
F02	Two stage	1/1	1/1	no/no	Unknown	Unknown; blocked
F03	Single stage	2/2	2/2	yes/yes	New submission	New submission
F03	Two stage	1/1	1/1	yes/yes	Same job reconciled	Same job reconciled

TABLE II
FOUR FIXED CASES, CHECKED AGAINST RETAINED REPORT FIELDS. S/E ARE PROXY SUBMISSIONS/BOUND EXECUTION INSTANCES; U/R ARE UNRESOLVED/RESOLVED RESEARCH-CLIENT CALLS. ALL FOUR CASES WERE ORACLE-EVALUABLE.

System	Case	Calls	S/E	Call sequence
B2	F00	1	1/1	R
W3	F00	1	1/1	R
B2	F02	2	2/2	U → R
W3	F02	2	2/2	U → R

TABLE III
TWO CASES PER SAME-RUN WINDOWS CAMPAIGN. C/S/E DENOTE PRODUCT CALLS, PROXY SUBMISSIONS AND BOUND BACKEND LIFECYCLES. THE F02 RETRY WAS REJECTED BY MODEL-IDENTITY VALIDATION IN V1 AND BY THE SUBMISSION GUARD IN V2. IDENTICAL COUNTS ALONE CANNOT DISTINGUISH THESE MECHANISMS.

Case / protocol	C	S	E	Client calls
W3 F00 / v1	1	1	1	resolved
W3 F00 / v2	1	1	1	resolved
W3 F02 / v1	2	1	1	unresolved / unresolved
W3 F02 / v2	2	1	1	unresolved / unresolved

complete generation arguments, then reused them verbatim after the controller reported a bindable first completion. The retained capture does not independently timestamp the retry invocation. The seed, operation key and run identifier were checked, along with before/after durable manifests. Table III reports this protocol separately from the four historical cases.

The F00 control completed. F02 suppressed one accepted response and retained an unresolved original manifest with unknown submission and no backend job. The second call

returned `model_identity_drifted` at generation, rather than the required `submission_outcome_unknown`. It generated no new POST and left the manifest byte-identical. The controller stopped with `submission_guard_observed=false`: another rejection masks the target mechanism, so withholding a submission cannot be attributed to that guard. The cause of the identity disagreement is unverified; its error code does not by itself establish changed model bytes.

A revised `same-run-guard-v2` caller rebuilt the pinned model inventory in both fresh MCP processes, using the existing exact-file index, fingerprint and private-catalog checks. It reused the complete generation arguments within the same run, without changing the frozen product or bypassing locked-route validation. In a separate two-case campaign, the F02 retry returned `submission_outcome_unknown` at generation. The controller reported `submission_guard_observed=true`, with one total F02-case POST and an unchanged unresolved manifest. The retry thus reached the target guard and was blocked in one fixed response-loss case. The original run was not reconciled, and eventual completion remains unestablished. The setup change supports a process-local inventory explanation for v1, without proving its entire internal error path.

Each campaign’s private capture retained 90 F00 and 10 F02 WebSocket text payloads and one history response per case. Offline reanalysis checked byte hashes and channel sequences, reconstructed one start and one `executing` event with `node=null` per case, and bound them to successful history and the published execution identifiers. This reanalysis checks the received, retained evidence without authenticating it or establishing completeness. Both controls and F02 cases included cache events. In F02 the cached-node list contained nodes 3–9; there were zero `progress` messages but one `progress_state` message, so its lifecycle binding is not evidence of new physical GPU computation. These new payloads do not recover the missing historical event lists. No third call or fresh-run fallback was made.

D. Resolution labels require an explicit aggregation rule

A resolved second call does not erase the first call’s unresolved result or show that its original run was reconciled. Read-only inspection of the retained campaign report confirmed that both F02 records carry `resolved=1` and `unresolved=1`. These are independent any-call flags: across the four cases, `resolved` sums to four and `unresolved` to two. They are overlapping counts, not mutually exclusive outcomes. The earlier Markdown summary’s zero unresolved cases is therefore not the controller aggregation. Table II preserves the call sequence; the evidence addendum records this correction without rewriting the historical report. No recovery-success rate is inferred.

V. REPRODUCIBILITY AND LIMITATIONS

The CPU records retain product revisions, harness digests and event traces. The companion ledger identifies Windows sources and reports. Audit and figure scripts reanalyse these records offline; reproducing a Windows campaign also requires its pinned backend, model, workflow and private setup. The historical four-case campaign was not rerun. Separate same-run campaigns preserved the frozen product and retained both the v1 failure and v2 observation.

Historical JSONL contains sanitized summaries, bindings, terminal labels and aggregate counts, without complete Web-

Socket payloads or history snapshots. Its audit checks consistency and regenerates two tables, but cannot reconstruct every backend event. The oracle requires `executing` with `node=null` and completed history. The exported terminal label lacks that discriminator and is set by construction, so checking it does not independently prove completion. Report/JSONL equality and image-byte matches remain historical inspection receipts rather than checks repeated by the offline script.

Host-side hashing matched the four generated-call artifact hashes to retained image bytes, with one product-output PNG per case. The first, unresolved F02 call recorded no product artifact hash; those checks therefore do not verify its output bytes. Oracle path fingerprints hash metadata rather than image content. Neither content hashes nor completed history measure visual quality or final user acceptance.

The observer must receive and bind events within its capture window. Missing events limit what the observer can evaluate; their absence does not prove non-execution. CPU and Windows observers have different failure boundaries; neither provides crash-complete auditability. The fixed cases support no population rates, performance gain or broad model robustness claim. The subsequent Windows campaigns used existing models and remain separate from historical observations. In particular, v2 supports one fixed-case guard observation, not an execution-count advantage. Private raw captures are excluded from delivered archives, limiting third-party reanalysis to the included evidence rather than the original received bytes.

The offline Windows conversion maps the retained product manifest separately from its audited backend lifecycle. F00 yields succeeded execution, partial evidence and no recovery requirement; F02 yields succeeded execution, missing evidence and required recovery. Neither is composite-verified: no independent artifact validator was supplied. Four variants (two missing-state and two job-identity-conflict) are explicitly synthetic checks, not additional Windows experiments. They retain unknown recovery or mismatch evidence. A clean-copy replay preserved all input bytes and reproduced the interpretation summary. The anonymous demo provides whitelisted, pseudonymized semantic projections whose interpretation states match the private conversion; it reproduces mapping, not raw-event audit.

VI. RELATED WORK AND DISCUSSION

Observation and control answer different questions at an uncertain submission boundary. A backend lifecycle can complete while the client still requires recovery; blocking a further submission does not discharge that obligation. RPC semantics and idempotent service contracts already address ambiguous outcomes and safe retries [1], [2]. Our tool instead records the evidence available when backend deduplication is unestablished. Neither a prompt identifier nor a client idempotency key alone supplies that guarantee.

Li’s preprint compares model, harness and tool-contract effects on duplicate side effects in a simulated service benchmark [3]. Its guard combines read-back, supported idemppo-

tency keys, delayed verification, blocking unverifiable repeats and uncertainty annotations. Our narrower guard blocks an explicitly unknown submission within one run, without automatic read-back or reconciliation. Withholding unsafe retries is therefore not our novelty claim. Our contribution is the local implementation that preserves reports and links their interpretation to backend observations. Different workloads and evidence sources preclude comparing the studies’ outcome counts.

Reliable State Machines use persistent state and atomic runtime processing to support failure transparency and exactly-once input processing [4]. Our external generator remains outside such a transactional runtime. Temporal likewise documents that completed Activities may be retried after a worker crashes before reporting completion, and recommends idempotent Activities [5]. These contracts explain why durable client state alone cannot settle external side effects. Our restricted submission entry provides neither equivalent workflow guarantees nor automatic reconciliation.

OpenTelemetry relates operation spans through trace context [6]. Our application-specific policy additionally requires backend start and terminal events bound to completed history, while unresolved recovery vetoes `execution_verified`. Correlation identifiers alone are not execution counts.

Toxiproxy injects configurable TCP connection faults [7]. Our proxy selects an accepted application response for suppression while a separate observer checks its lifecycle. No comparative implementation or usability study was performed, so equivalent Toxiproxy setups are not ruled out. Filibuster’s project overview describes synthesizing fault-injected variations from integration tests [8]. Our fixed submission cases offer no such search coverage; this comparison uses the overview, not reproduced Filibuster results.

These comparisons use the documented contracts to define each tool’s responsibilities. They establish no superiority, exhaustive novelty or developer productivity gain. Within the evaluated workflow, our records let a reader inspect the client’s ambiguous outcome, the backend observation and the claims those observations support.

VII. CONCLUSION

The tool records ambiguous client outcomes alongside bound backend observations and restricts further submissions through an unresolved run. Historical Windows cases distinguish a successful fresh-run call from original-run reconciliation. Same-run tests retain both a masking failure and one fixed-case guard observation. The original run remains unresolved, and blocking another submission supplies no physical GPU-savings evidence. Automatic unknown-job recovery, deployment reliability and natural duplicate-execution rates remain unestablished.

DATA, CODE AND RESPONSIBLE USE

The manuscript source includes derived Windows evidence, retained CPU-record pointers, a deterministic audit script and figure sources. Models, private configuration, generated

images and raw host paths are excluded from the external anonymous demo. A separate private author-review bundle contains exact source snapshots and audit metadata with host/account identifiers and local paths; it is not an anonymous or public distribution. The analysis concerns controlled software executions; it makes no claim about human participants or image-quality acceptance. Human-subjects or institutional-ethics determinations are not inferred from this scope.

AI TOOL USAGE

OpenAI Codex assisted with drafting portions of the manuscript, source inspection, record consistency checks, and code for Figs. 1–2. The Humanizer editing skill was used for prose revision. The review conducted with this tool included separate and continued review contexts within the same model family, not independent peer review. Human authors must verify the content, references, attribution, and final submission. No AI-generated data are reported as experimental observations.

REFERENCES

- [1] A. D. Birrell and B. J. Nelson, “Implementing remote procedure calls,” *ACM Transactions on Computer Systems*, vol. 2, no. 1, pp. 39–59, 1984. [Online]. Available: <https://www.cs.cmu.edu/~15712/papers/birrell84.pdf>
- [2] M. Featonby, “Making retries safe with idempotent apis,” Amazon Builders’ Library, accessed September 27, 2026. [Online]. Available: <https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/>
- [3] J. Li, “Where does exactly-once live? model, harness, and tool-contract effects on duplicate side effects in LLM agents,” arXiv preprint arXiv:2609.29095v1, 2026, version 1, September 24, 2026. [Online]. Available: <https://arxiv.org/abs/2609.29095v1>
- [4] S. Mukherjee, N. J. Raj, K. Govindraj, P. Deligiannis, C. Ravichandran, A. Lal, A. Rastogi, and R. Krishnaswamy, “Reliable state machines: A framework for programming reliable cloud services,” in *33rd European Conference on Object-Oriented Programming (ECOOP 2019)*, ser. Leibniz International Proceedings in Informatics, vol. 134, 2019, pp. 18:1–18:29. [Online]. Available: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2019.18>
- [5] Temporal Technologies, “Activity definition,” official documentation. Accessed September 27, 2026. [Online]. Available: <https://docs.temporal.io/activity-definition>
- [6] OpenTelemetry Authors, “Traces,” official documentation. Accessed September 27, 2026. [Online]. Available: <https://opentelemetry.io/docs/concepts/signals/traces/>
- [7] Shopify, “Toxiproxy,” official project documentation. Accessed September 27, 2026. [Online]. Available: <https://github.com/Shopify/toxiproxy>
- [8] Filibuster, “Reliability starts with the application,” official project overview. Accessed September 27, 2026. [Online]. Available: <https://filibuster.cloud/>